COMPUTER SCIENCE + COMPUTER ENGINEERING

Answers and review guidance

Open after an independent attempt

Explanatory quiz answers, worked exercise guidance and practical review standards. Use mistakes to identify the next concept to repair.

125 quiz/review answers · 50 exercise solutions and evidence standards

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Contents

Use the linked contents or PDF bookmarks to navigate.

How to use the answer key 3

Module 01: Starting as an engineer 4
- Lesson 01.1: What a program actually does 4
- Lesson 01.2: Files, terminals and reproducible execution 5
- Lesson 01.3: Trace before running 6
- Lesson 01.4: Units, budgets and engineering notebooks 7
- Module 01: additional-exercise solutions 8
- Module 01: review answers and project evaluation 10

Module 02: Values, decisions and repetition 12
- Lesson 02.1: Types and expressions 12
- Lesson 02.2: Conditions and boundary cases 13
- Lesson 02.3: Loops and invariants 14
- Lesson 02.4: Functions and contracts 15
- Module 02: additional-exercise solutions 16
- Module 02: review answers and project evaluation 18

Module 03: Organizing data and small programs 20
- Lesson 03.1: Sequences, dictionaries and identity 20
- Lesson 03.2: Decomposition, modules and interfaces 21
- Lesson 03.3: Files, serialization and validation 22
- Lesson 03.4: Classes, composition and state machines 23
- Module 03: additional-exercise solutions 24
- Module 03: review answers and project evaluation 26

Module 04: Debugging, testing and collaboration 28
- Lesson 04.1: Debugging as an experiment 28
- Lesson 04.2: Unit, integration and property tests 29
- Lesson 04.3: Git, branches and recoverable changes 30
- Lesson 04.4: Review, documentation and responsible AI help 31
- Module 04: additional-exercise solutions 32
- Module 04: review answers and project evaluation 34

Module 05: A complete first game 36
- Lesson 05.1: The update loop and command model 36
- Lesson 05.2: Collision rules and complete behavior 37
- Lesson 05.3: Deterministic replay and simple measurement 38
- Lesson 05.4: Finishing, usability and the first portfolio artifact 39
- Module 05: additional-exercise solutions 40
- Module 05: review answers and project evaluation 42

How to use the answer key

Attempt the lesson, changed example and quiz first. Score each quiz answer 0-2: absent/incorrect, partial or unexplained, correct with reasoning. Core lesson questions 1-3 need at least 5/6; all five need at least 8/10. Module reviews use a suggested 8/10 threshold. Correct answers alone do not satisfy lab or project requirements.

This key includes explanatory answers to 100 lesson questions and 25 module-review questions, worked solutions/guidance for 50 additional exercises, and practical review notes. The first 60 lesson questions/answers are retained from the original curriculum; the two application questions per lesson and all module reviews are new. Model answers are not the only acceptable wording: accept equivalent technically sound reasoning with explicit assumptions.

For each missed idea, identify the misconception, solve a changed problem and recheck later. When an answer requires local evidence or an implementation, do not replace it with an invented environment or a stock paragraph. A mentor should inspect the actual artifact and ask for a new variation.

The complete reference implementations are in labs/reference/ and capstone/reference/. The student's own code must remain separate. Reviewing a reference after an attempt is useful; submitting it unchanged is not evidence of independent completion.

Module 01: Starting as an engineer

Lesson 01.1: What a program actually does

Core question 1

Question: What is the difference between state and a displayed image?

Answer and reasoning: State is the underlying information; the image is one view of that information.

Core question 2

Question: What final position results from the five lab commands?

Answer and reasoning: 2 + 3 - 1 + 0 - 4 + 2 = 2.

Core question 3

Question: Why should drawing a character not update its position?

Answer and reasoning: Rendering may occur at a different rate or multiple times; movement must follow simulation rules, not draw count.

Application question 4

Question: A game stores health=10 and a label reading "10". Damage changes health to 7 but not the label. Which information should normally be authoritative in this first game, and how would you prevent disagreement?

Answer and reasoning: Treat the numeric health as authoritative and generate the label from it when drawing. Do not maintain two independently editable sources of truth.

Application question 5

Question: Two implementations finish at x=2. Is that sufficient to show they applied the five commands correctly?

Answer and reasoning: No. Compare intermediate states or independently test individual transitions. For example, an extra +1 followed by an extra -1 can hide two defects in the final total.

Practical review guidance

Start on paper at x=2. Trace right 3, left 1, right 0, left 4, right 2. Include the initial state and all five following states. Mark which operations are reads and which are writes. Then run `examples/example_01_1.py` twice and compare its smaller demonstration with your prediction.

Create `workspace/01_1_trace.md`. Include the six-position trace, a plain-language update rule, and an example where two wrong movements cancel. Your pass evidence is an explanation of every transition and why drawing twice must not move twice. The optional function scaffold `labs/student/lab_01_1.py` is a return visit after Lesson 02.4; functions and loops are not prerequisites for this paper-first lesson.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 01.2: Files, terminals and reproducible execution

Core question 1

Question: What does a relative filename depend on?

Answer and reasoning: The process working directory, unless the application explicitly resolves it another way.

Core question 2

Question: Is cd a Python statement in an ordinary script?

Answer and reasoning: No. It is normally a shell command; Python has separate filesystem APIs.

Core question 3

Question: What three facts help another person reproduce a run?

Answer and reasoning: The exact source revision/file, tool environment and command; inputs and working directory also matter.

Application question 4

Question: You are in the package root, but type `python3 example_01_1.py`. The file is inside examples/. What is the smallest likely repair?

Answer and reasoning: Use the correct relative path, `python3 examples/example_01_1.py`, or deliberately change to the examples directory. First verify that the file actually exists at the expected location.

Application question 5

Question: A saved edit produces no change in output. Give two checks before reinstalling anything.

Answer and reasoning: Confirm the editor saved the file you are executing, and confirm the command/working directory resolves to that same file. Also inspect whether the changed line is actually reached.

Practical review guidance

Run `tools/environment_check.py` and save the output with your actual command in `workspace/01_2_environment.md`. Run the movement example, then deliberately change only its filename in the command to a name that does not exist. Record the resulting error, identify which program emitted it and repair the command.

Repeat the valid command from the package root. Your evidence must distinguish a script path from the working directory and explain how a relative filename is resolved. This is an evidence-based lab: no Python function starter is assigned and the automatic lab checker does not certify environment understanding.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 01.3: Trace before running

Core question 1

Question: After a = 3; b = a; a = 8, what is b for these integers?

Answer and reasoning: 3. The earlier integer value was bound to b; changing a does not recompute b.

Core question 2

Question: Why does x = y; y = x fail to swap two different values?

Answer and reasoning: The first assignment overwrites the old x, so both names end with the old y.

Core question 3

Question: What makes prediction useful when a run disagrees?

Answer and reasoning: It exposes the specific mistaken assumption, which can then be tested instead of hidden by random edits.

Application question 4

Question: Trace `a=2; b=a*4; a=b-1; b=a+2`. What are a and b at the end?

Answer and reasoning: After the statements: a=2; b=8; a=7; b=9. Final a=7 and b=9.

Application question 5

Question: Why should you preserve your incorrect prediction instead of rewriting it to match the output?

Answer and reasoning: The disagreement identifies the mistaken assumption. Rewriting the prediction erases the evidence needed to repair and later recognize that reasoning error.

Practical review guidance

Create three four-statement assignment sequences in `workspace/01_3_trace.md`. Include an incorrect swap, the repaired swap and a sequence in which a derived value does not update automatically. For each, predict every binding and final printed output before checking with Python.

The optional starter `labs/student/lab_01_3.py` packages the swap in a function; return to it after Lesson 02.4. Pass now by explaining where the incorrect swap loses the old value and by solving a fresh sequence without running it first. Do not count a memorized one-line swap as sufficient explanation.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 01.4: Units, budgets and engineering notebooks

Core question 1

Question: How far does 8 units/second travel in 0.125 seconds?

Answer and reasoning: 1 unit, because 8 multiplied by 0.125 is 1.

Core question 2

Question: What is the interval for 120 frames/second?

Answer and reasoning: 1000/120 = approximately 8.33 milliseconds.

Core question 3

Question: Why record hardware and workload with a timing result?

Answer and reasoning: The result depends on the environment and work performed; without them it cannot be meaningfully reproduced or generalized.

Application question 4

Question: A serial frame has 2 ms input work, 5 ms simulation and 11 ms drawing. Does that model fit a 60-frame/s interval?

Answer and reasoning: The total is 18 ms, exceeding approximately 16.67 ms by 1.33 ms. This conclusion applies to the stated serial model, not an unmeasured overlapped pipeline.

Application question 5

Question: Why can a finite-value check accept a calculation that is wrong by a factor of 1,000?

Answer and reasoning: A milliseconds-versus-seconds mistake may still use ordinary finite numbers. Type/range checks do not establish correct units or conventions.

Practical review guidance

Calculate all six combinations of speeds 2, 6 and 10 units/second with intervals 0.1 and 0.5 seconds. Then calculate the frame intervals for 30, 60 and 120 frames/second. Show units and identify the chosen targets. Store the table in `workspace/01_4_units.md`.

After functions are introduced, implement the `displacement` and `frame_interval_ms` starter contracts. Their input domain is finite numeric values, with nonnegative elapsed time and positive frame rate; finite overflow results must also be rejected. These programmatic checks supplement rather than replace the written unit analysis.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Module 01: additional-exercise solutions

M01-E01

Exercise: Trace start=7 with changes -2,+5,-8,+1. Include the initial state.

Worked answer or evidence standard: The state sequence is 7,5,10,2,3. There are four commands and five snapshots.

M01-E02

Exercise: Construct two wrong intermediate movements that preserve the correct final position. Explain why final-only testing misses them.

Worked answer or evidence standard: For example add an unintended +2 on one step and -2 on a later step. The final error cancels, but the intermediate traces disagree.

M01-E03

Exercise: Classify a keypress, player position, drawing function and monitor image as input, state, rule/operation or output.

Worked answer or evidence standard: Keypress is input; stored position is state; drawing is an operation observing state; the image is output. Drawing is not the movement rule.

M01-E04

Exercise: From a parent folder, explain how to run a script in Game_Engineering_Package_02/ examples without changing folders.

Worked answer or evidence standard: Use its full relative path from that parent, or an absolute path. The working directory remains the parent unless explicitly changed.

M01-E05

Exercise: Trace a=5; b=a+1; a=b+1; c=a+b.

Worked answer or evidence standard: a becomes 5, b becomes 6, a becomes 7, c becomes 13.

M01-E06

Exercise: Explain why swapping equal values is a weak test of a swap algorithm.

Worked answer or evidence standard: A broken overwrite-based swap can appear correct when both inputs are equal. Use different values as well.

M01-E07

Exercise: Convert 16 milliseconds to seconds and calculate travel at 2.5 units/second.

Worked answer or evidence standard: 16/1000=0.016 seconds; 2.5*0.016=0.04 units.

M01-E08

Exercise: Calculate the target interval at 144 frames/second. Label it as a target or measurement.

Worked answer or evidence standard: 1000/144 is approximately 6.944 milliseconds per frame. It is a target interval derived from a chosen rate, not measured hardware performance.

M01-E09

Exercise: Write a reproducibility recipe for one example using your actual environment.

Worked answer or evidence standard: Require source identity, interpreter, working directory, command, inputs, expected and actual output. Environment-specific answers should not be replaced with invented paths.

M01-E10

Exercise: Explain the distinction between negative velocity and negative elapsed time in this course.

Worked answer or evidence standard: Negative velocity is allowed to indicate direction. Negative dt violates the selected update contract; this course function does not implement time reversal.

Module 01: review answers and project evaluation

M01-R01

Question: Why can rendering the same world from two views expose a poor state/update boundary?

Answer: A renderer that changes state can update twice simply because two views are drawn. Correct observation leaves state unchanged.

M01-R02

Question: A script file exists but cannot be found with a relative path. What fact must be checked?

Answer: The working directory and the path relative to it; also verify the actual file name.

M01-R03

Question: What are x and y after x=3; y=x+4; x=y*2?

Answer: x=14 and y=7.

M01-R04

Question: At 30 frames/s, does a stated 40 ms serial frame fit the interval?

Answer: No. The target interval is about 33.333 ms, and 40 ms exceeds it by about 6.667 ms.

M01-R05

Question: What evidence lets a reviewer distinguish a prediction from an observation?

Answer: Preserve both separately with the planned calculation written before the run, then record the actual result and the explanation of any difference.

Project acceptance guidance

A reviewer can repeat the command, match the trace, identify the working directory and hear an explanation of why two renders do not cause two updates. Complete the first-module paper work before using the optional function starters after 02.4.

Require the common rubric, essential failure checks and an explainable new variation. Independently observed results must be identified separately from self-reported results.

Module 02: Values, decisions and repetition

Lesson 02.1: Types and expressions

Core question 1

Question: Why are "12" and 12 different?

Answer and reasoning: One is text and one is an integer; their operations and validation rules differ.

Core question 2

Question: What does "ab" + "cd" produce?

Answer and reasoning: The string "abcd".

Core question 3

Question: Would an exact item count normally use floating point? Why?

Answer and reasoning: No. An integer directly represents whole counts and avoids fractional or approximation ambiguity.

Application question 4

Question: Why is `"-2"` a different failure case from `"two"` in a nonnegative item calculator?

Answer and reasoning: `"-2"` parses successfully as an integer and then violates the allowed range. `"two"` cannot be parsed as an integer in this interface.

Application question 5

Question: A programmer fixes text input by concatenating and then converting: `int("3" + "4")`. Does that calculate the sum or product of 3 and 4?

Answer and reasoning: Neither. It produces integer 34 from concatenated text. Convert the separate inputs before applying the intended arithmetic operation.

Practical review guidance

Implement `total_cost(count_text, unit_cost_text)` in `labs/student/lab_02_1.py`. Use whole-number text accepted by Python's integer conversion; reject malformed text and negative values with `ValueError`. Return the product without printing. Test count zero, price zero, ordinary values, surrounding spaces, negative values and fractional text.

Run `tools/check_labs.py --lesson 02.1` with your selected Python launcher. Also create a small interaction script that asks for two strings, calls the function and catches only the expected input error. Save an input/output table. Passing the supplied checks is necessary for the function exercise, but you must explain parsing versus domain validation independently.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 02.2: Conditions and boundary cases

Core question 1

Question: Does gold > price allow buying an item when gold equals price?

Answer and reasoning: No. Use >= when exact funds are sufficient.

Core question 2

Question: Evaluate False or (True and False).

Answer and reasoning: False: the parenthesized conjunction is false, and neither side of or is true.

Core question 3

Question: Why test immediately below, at and above a threshold?

Answer and reasoning: Those cases expose incorrect inclusive/exclusive comparisons and missing boundary rules.

Application question 4

Question: For key=true, level=0 and stamina=0, compare the assigned expression with `(has_key or level >= 5) and stamina >= 10`.

Answer and reasoning: The assigned expression returns true because the key is sufficient. The altered expression returns false because it requires stamina >= 10 even for a key holder.

Application question 5

Question: Why is a nonempty string such as `"false"` unsafe to use directly as the Boolean key flag?

Answer and reasoning: Nonempty strings are truthy in Python. A text interface must parse allowed Boolean spellings into actual Boolean values; it must not rely on the appearance of the word.

Practical review guidance

Implement `can_open` in `labs/student/lab_02_2.py`. Prepare the 18-row table using key=false/true, level=4/5/6 and stamina=9/10/11. Add the key=true, level=1, stamina=0 counterexample. Explain why an expression requiring stamina for every key holder would fail that case.

Run the lesson checks and write one caller that prints an open/closed message based on the returned Boolean. Do not deduct stamina in the predicate. Pass by defending the grouping, the inclusive comparisons and every table row, not just by observing a green test run.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 02.3: Loops and invariants

Core question 1

Question: How many values does range(2, 7) produce?

Answer and reasoning: Five: 2, 3, 4, 5 and 6.

Core question 2

Question: What must change in while remaining > 0 to make progress?

Answer and reasoning: The body must eventually reduce remaining or otherwise make the condition false, unless it exits explicitly.

Core question 3

Question: Give an invariant for an inventory limited to ten items.

Answer and reasoning: At the chosen loop boundary, the item count is always between zero and ten inclusive.

Application question 4

Question: How many entries should the returned trace contain for zero turns and for ten turns under this lesson's convention?

Answer and reasoning: One entry for zero turns: the initial state. Eleven entries for ten turns: the initial state plus ten results.

Application question 5

Question: Does “resources never become negative” prove that exactly ten turns were simulated?

Answer and reasoning: No. A program doing no updates could preserve nonnegativity. Check both state invariants and progress/count requirements.

Practical review guidance

Implement `simulate_resources(turns)` in `labs/student/lab_02_3.py`. Return the initial zero plus the value after every complete turn. Reject negative, non-integer and Boolean turn counts. The strict Boolean exclusion is a schema choice: a yes/no flag is not an acceptable turn count even though Python relates bool and int.

Trace ten turns manually and compare all eleven values. Test zero turns. Explain the nonnegative invariant and show a changed example that breaks an unconditional-spending version. Run the supplied checks, then create at least two additional tests with turn counts not used in the demonstration.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 02.4: Functions and contracts

Core question 1

Question: What is the result of advance(2, 8, 0.25)?

Answer and reasoning: 4, because 2 + 8 times 0.25 equals 4.

Core question 2

Question: Why is returning a value more reusable than only printing it?

Answer and reasoning: Other code can consume and test the result without parsing console output.

Core question 3

Question: What belongs in a function contract besides parameter names?

Answer and reasoning: Valid ranges/types, units, outputs, side effects and failure behavior.

Application question 4

Question: A function prints 12 but has no explicit return. What does `answer = function()` normally bind to answer?

Answer and reasoning: It binds None. Printing a value and returning a value are separate operations.

Application question 5

Question: Why check the result for finiteness when all three inputs are finite?

Answer and reasoning: Finite floating-point inputs can overflow during multiplication or addition. The contract promises a finite result, so that property must also be checked.

Practical review guidance

Implement the documented function in `labs/student/lab_02_4.py`. Test normal movement, zero velocity, negative velocity, zero dt, negative dt, infinity, not-a-number and an overflowing result. Capture or inspect output to confirm the function itself does not print.

Write a contract comment naming units and side effects. Then revisit the optional first-module function starters and run their checks. Pass by explaining the difference between returning and printing, and by showing that a caller can retain its old value after catching an invalid update request.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Module 02: additional-exercise solutions

M02-E01

Exercise: Predict "8"+"2", int("8")+int("2") and int("8")*int("2").

Worked answer or evidence standard: The results are text "82", integer 10 and integer 16.

M02-E02

Exercise: Add a count ceiling of 99 to a copied calculator. Choose three boundary inputs.

Worked answer or evidence standard: Use 98,99,100 with expected accept,accept,reject. Preserve zero and malformed-input cases too.

M02-E03

Exercise: Compare two key-door groupings using a key holder with stamina zero.

Worked answer or evidence standard: The assigned has_key or (level>=5 and stamina>=10) accepts. (has_key or level>=5) and stamina>=10 rejects.

M02-E04

Exercise: Explain why level>5 is not equivalent to level>=5.

Worked answer or evidence standard: They differ at exactly 5. A boundary test must include the threshold itself.

M02-E05

Exercise: Trace three resource turns with the collect-three/spend-five rule.

Worked answer or evidence standard: Including initial state: 0,3,1,4.

M02-E06

Exercise: Change collection to seven and explain which invariant survives.

Worked answer or evidence standard: Nonnegative balance survives conditional affordable spending. Balance below five no longer necessarily holds after a single subtraction.

M02-E07

Exercise: Write a while-loop example that never changes the condition, then describe the repair without running an infinite loop.

Worked answer or evidence standard: An example remaining=3; while remaining>0: print(remaining) lacks progress. Decrement remaining or use a bounded for loop; include a deliberate exit policy.

M02-E08

Exercise: Design two callers for advance: one prints and one tests.

Worked answer or evidence standard: Both call the same pure function. One formats its returned value; the other compares it with an independently derived expectation.

M02-E09

Exercise: Test a movement with x=10, velocity=-4, dt=0.5.

Worked answer or evidence standard: The new position is 8. Zero dt keeps 10; negative dt must be rejected.

M02-E10

Exercise: Show how a caller preserves a previous position after advance rejects dt.

Worked answer or evidence standard: Assign from the call inside try and catch the expected ValueError. Since the failed call never returns, the assignment does not replace the previous valid binding.

Module 02: review answers and project evaluation

M02-R01

Question: What is the distinction between parsing text and accepting a game-domain value?

Answer: Parsing interprets representation; domain validation checks allowed types, ranges and relationships. Parsed -3 can still be forbidden as a count.

M02-R02

Question: How many local door-boundary combinations use two key flags and three values for each numeric threshold?

Answer: $2^3 3 = 18$ combinations.

M02-R03

Question: Why does nonnegative inventory alone not prove a loop ran the correct number of turns?

Answer: A no-op or too-short loop can preserve the invariant. Count/progress requirements must also be tested.

M02-R04

Question: What does return allow that printing alone does not?

Answer: It gives a value directly to the caller for storage, composition or testing without parsing displayed text.

M02-R05

Question: For the five-turn project sequence, explain the last rejected buy.

Answer: Before the final buy there is one resource and health 10. Buying requires five, so state stays unchanged, but the recognized attempt still occupies the fifth turn slot under this project policy.

Project acceptance guidance

The fixed sequence ends with health 10 and resources 1; the full trace is retained. Test caps, zero turns in a reusable driver, invalid text, exact affordability and unaffordable purchase. Explain the project-specific turn policy rather than borrowing one accidentally.

Require the common rubric, essential failure checks and an explainable new variation. Independently observed results must be identified separately from self-reported results.

Module 03: Organizing data and small programs

Lesson 03.1: Sequences, dictionaries and identity

Core question 1

Question: When is a dictionary preferable to a list?

Answer and reasoning: When the main operation is association or lookup by a stable key rather than by position.

Core question 2

Question: Does b = a copy a Python list?

Answer and reasoning: No. Both names refer to the same list object.

Core question 3

Question: Why can a shallow copy still share changes?

Answer and reasoning: Its nested objects can remain the same mutable objects referenced by the original.

Application question 4

Question: After `b = list(a)`, where a contains dictionaries, does changing `b[0]["health"]` necessarily leave a unchanged?

Answer and reasoning: No. The outer list is copied, but both lists still refer to the same dictionary elements. Copy or otherwise isolate the records when that is the intended contract.

Application question 5

Question: Two units share the name Scout but have IDs 101 and 102. Must an ID index reject them?

Answer and reasoning: No. Names may duplicate under this schema. It must reject duplicate IDs, because IDs are the unique lookup key.

Practical review guidance

Create five flat unit records and implement `index_units` and `living_units` in `labs/student/lab_03_1.py`. Use stable IDs, reject duplicate IDs and return independent shallow copies of flat records. Preserve the input collection while filtering.

Test an empty collection, all-dead units, duplicate names with different IDs and duplicate IDs. Rename a copied living unit and demonstrate that the original record is unchanged. Explain why the same technique would not isolate a nested inventory list. Keep the small input data and the before/after snapshots in your journal.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 03.2: Decomposition, modules and interfaces

Core question 1

Question: What is an interface?

Answer and reasoning: The supported operations and contracts through which another part of the system interacts with a component.

Core question 2

Question: Why avoid starting a game as a side effect of importing simulation.py?

Answer and reasoning: Imports should make reusable definitions available; automatic interaction prevents tests and other tools from using them cleanly.

Core question 3

Question: How does command-as-data design help replays?

Answer and reasoning: Recorded commands can be fed back through the same simulation interface.

Application question 4

Question: Why can placing `input()` at module top level make automated tests hang?

Answer and reasoning: Importing executes top-level statements. A test importing the module can therefore start waiting for terminal input before it reaches its assertions.

Application question 5

Question: A refactor changes a rejected purchase from no change to a negative balance. Is that merely an internal restructuring?

Answer and reasoning: No. It changes observable behavior and violates the assigned nonnegative-balance contract. Keep structural changes and deliberate rule changes separate.

Practical review guidance

Implement the strict `spend` contract in `labs/student/lab_03_2.py`. Refactor your earlier resource simulator into a rule module and a separate command-line entry point in `workspace/module03_simulator/`. The rule module must be importable without output or prompts.

Save a before-refactor trace and compare it with the after-refactor trace. Add direct function tests for exact affordability, insufficient balance, zero cost and invalid input. Pass by drawing the dependency direction and showing an independent test that uses the rule without launching the interface.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 03.3: Files, serialization and validation

Core question 1

Question: Does valid JSON imply a valid game state?

Answer and reasoning: No. Syntax validity does not establish schema, range or game-rule validity.

Core question 2

Question: What does a round-trip test check?

Answer and reasoning: That the intended data survives serialization and deserialization without unacceptable change.

Core question 3

Question: Why include a format version?

Answer and reasoning: It lets the loader detect compatibility requirements and choose migration or a clear rejection.

Application question 4

Question: Why is `{"version":1,"x":2,"health":-50}` invalid even though it is valid JSON?

Answer and reasoning: The structure parses, but health violates the game schema range 0..100. Syntax validity does not establish domain validity.

Application question 5

Question: What is the danger of resetting the live world to an empty dictionary before attempting a load?

Answer and reasoning: A parse or validation failure then leaves the application without its previous valid world. Build and validate a candidate first, then replace live state only on success.

Practical review guidance

Implement `encode_save` and `decode_save` in `labs/student/lab_03_3.py`, including the exact schema described above, duplicate-key rejection and the input-length limit. Return new dictionaries and do not write files inside these functions. Use a separate disposable workspace file to exercise a file wrapper.

Test a valid round trip, missing and extra fields, wrong types, a Boolean version, unsupported version, out-of-range health, duplicate keys, excessive text and truncated input. Keep the known-good state and save file unchanged after a failed load. Pass only after explaining why successful JSON parsing is not the same as valid game data.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 03.4: Classes, composition and state machines

Core question 1

Question: What does encapsulation protect?

Answer and reasoning: An object's invariants by controlling how its state can be changed.

Core question 2

Question: Why can independent booleans create contradictory states?

Answer and reasoning: They allow combinations that may not correspond to any valid behavior unless separately constrained.

Core question 3

Question: When is composition useful?

Answer and reasoning: When independent capabilities should be combined without forcing them into one rigid inheritance tree.

Application question 4

Question: Under the assigned table, what happens to a stunned controller receiving move, followed by recover?

Answer and reasoning: Move is unlisted for stunned, so it remains stunned. Recover then transitions it to idle.

Application question 5

Question: Why test two controller instances instead of only one?

Answer and reasoning: It can expose accidentally shared state or mutable defaults. A correct isolated transition does not prove that one character's changes are independent of another's.

Practical review guidance

Implement `transition` in `labs/student/lab_03_4.py` with the exact five-entry table and the stated no-op/error policy. Build a small controller wrapper in your workspace and add an inventory field by composition rather than a hierarchy of movement subclasses.

Test every allowed transition, at least three forbidden transitions and an unknown state. Create two controller instances and demonstrate independence. Trace move, hit, move, recover, move, stop from idle. Pass by explaining both the state table and which object or function owns each change.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Module 03: additional-exercise solutions

M03-E01

Exercise: Create two units with the same name but distinct IDs and show stable lookup.

Worked answer or evidence standard: The index contains separate entries keyed by each ID. Name uniqueness is not required by the schema.

M03-E02

Exercise: Demonstrate outer-list copying without record copying.

Worked answer or evidence standard: b=list(a) creates a new list but shares dictionary elements; editing b[0]["health"] can affect a[0].

M03-E03

Exercise: Demonstrate why a flat-record copy does not isolate a nested inventory list.

Worked answer or evidence standard: dict(record) copies the dictionary entries, but an inventory list remains a shared referenced object unless separately copied or made immutable.

M03-E04

Exercise: Filter five units without deleting from the source during iteration.

Worked answer or evidence standard: Build a new collection, retaining only health>0. Preserve the original ordering and records according to the documented copy policy.

M03-E05

Exercise: Draw the dependency direction between interface and simulation.

Worked answer or evidence standard: The interface imports/calls the simulation. The simulation does not prompt or import the terminal entry point to execute a rule.

M03-E06

Exercise: Create a syntactically valid JSON save that fails the game schema.

Worked answer or evidence standard: For example {"version":1,"x":2,"health":101}; health exceeds the maximum despite valid JSON syntax.

M03-E07

Exercise: Explain why true is rejected as version 1 by the strict schema.

Worked answer or evidence standard: The format requires an actual integer version, not a Boolean flag; exact type checks avoid treating the bool/int relationship as permission.

M03-E08

Exercise: Specify unknown-field behavior for a future configuration version.

Worked answer or evidence standard: A sound answer states reject, preserve or ignore behavior explicitly, identifies its compatibility consequences and tests the policy rather than silently accepting all fields.

M03-E09

Exercise: Trace move, hit, move, recover, move, stop from idle.

Worked answer or evidence standard: States after events: moving,stunned,stunned,idle,moving,idle.

M03-E10

Exercise: Create two controllers and prove one can move without changing the other.

Worked answer or evidence standard: Initialize state per instance, send an event to one, and assert that the other remains idle. Explain the absence of shared mutable state.

Module 03: review answers and project evaluation

M03-R01

Question: Why is a list index usually a weak long-lived entity identifier?

Answer: Sorting or removal can change which entity occupies that index. A stable ID is independent of the current ordering.

M03-R02

Question: What does the main guard prevent, and what does it not prevent?

Answer: It prevents the guarded entry-point call during an ordinary import. Other top-level statements still execute.

M03-R03

Question: Why is successful JSON parsing insufficient for loading a game?

Answer: The parsed object may have wrong keys, types, ranges, versions or inconsistent field relationships.

M03-R04

Question: What happens to live state when candidate decoding fails before assignment?

Answer: With candidate-first assignment and handled failure, the previous valid live binding remains available.

M03-R05

Question: Why use composition for inventory and movement state?

Answer: They represent potentially independent concerns. Separately owned components avoid an unnecessary class for every combination of inventory and movement mode.

Project acceptance guidance

A clean run reads valid configuration, accepts a saved round trip, rejects malformed/unsupported data and preserves live state after failure. Imports do not prompt. A changed unit name does not invalidate ID lookup. Demonstrate all five allowed controller transitions and three forbidden ones.

Require the common rubric, essential failure checks and an explainable new variation. Independently observed results must be identified separately from self-reported results.

Module 04: Debugging, testing and collaboration

Lesson 04.1: Debugging as an experiment

Core question 1

Question: What information makes a bug report actionable?

Answer and reasoning: Reproduction steps, inputs/environment, expected behavior, actual behavior and useful evidence.

Core question 2

Question: Why inspect the earliest wrong state?

Answer and reasoning: The final crash may be a downstream symptom; the first deviation is closer to the cause.

Core question 3

Question: What is a regression test?

Answer and reasoning: A test that preserves a previously failing case to detect reintroduction of that defect.

Application question 4

Question: A correct damage function returns 7 for health 10 and damage 3, but the whole game reaches 4. What layer should you investigate next?

Answer and reasoning: Investigate event dispatch and call count, including whether update and drawing both invoke damage. A correct function may be called twice.

Application question 5

Question: Why should a regression test fail before the fix is applied?

Answer and reasoning: That establishes that the test actually exposes the reported defect. A test that already passes cannot demonstrate that this particular repair corrected the observed failure.

Practical review guidance

Introduce a double-damage bug into a disposable copy of your simulator. Write a minimal report with health 10, damage 3, expected 7 and actual behavior. Trace or break at the first incorrect update. Add a failing test before repairing the copied simulator.

Implement `apply_damage` in `labs/student/lab_04_1.py`: nonnegative integer inputs, one subtraction and a zero floor. Demonstrate that the repaired version passes, and that temporarily restoring the known bug makes a relevant test fail. The supplied `tools/mutation_demo.py` illustrates this method; your own bug report and regression test remain required.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 04.2: Unit, integration and property tests

Core question 1

Question: How does an integration test differ from a unit test?

Answer and reasoning: It checks cooperation between components rather than one isolated responsibility.

Core question 2

Question: Why is copying the implementation into the test oracle weak?

Answer and reasoning: The test can reproduce the same mistake and falsely agree with the implementation.

Core question 3

Question: What does a test suite passing establish?

Answer and reasoning: The exercised checks passed in that environment; it does not prove every requirement or input is correct.

Application question 4

Question: Why is a bounds-only property insufficient for a movement function?

Answer and reasoning: A function that never moves can remain in bounds while violating valid movement requests. Include progress and exact transition expectations as well.

Application question 5

Question: Does `--reference` on the supplied checker validate your edited student files?

Answer and reasoning: No. It selects the separate completed reference implementations. Run without --reference to check the student directory, and add your own tests.

Practical review guidance

Implement `bounded_move` in `labs/student/lab_04_2.py` with the documented whole-move rejection policy. The starting size must be positive and the starting position inside the board. Then write at least twelve learner-owned tests in `workspace/test_my_rules.py` across motion, spending and save/load, including at least one bounded property loop.

Record each test's requirement and independently derived expected result. Introduce an off-by-one or double-update mutation in a disposable copy and show a named test detects it. Restore the correct code. A complete submission includes tests, a successful run and the mutation experiment, not only the supplied checker output.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 04.3: Git, branches and recoverable changes

Core question 1

Question: What does the staging area control?

Answer and reasoning: Which current file changes become part of the next commit.

Core question 2

Question: Why should secrets not be committed even if later deleted?

Answer and reasoning: Earlier commits may still retain them and copies may already exist.

Core question 3

Question: What makes a useful commit message?

Answer and reasoning: It identifies the behavior or purpose of the change and, when needed, its reasoning.

Application question 4

Question: You stage version A of a file, edit it to version B, and commit without staging again. Which content is committed?

Answer and reasoning: The staged content, version A. The later edit remains an unstaged working-tree change unless another operation stages it.

Application question 5

Question: Does reverting a mistaken commit normally remove that commit from the project history?

Answer and reasoning: No. Revert creates a new commit reversing the selected change. The original commit remains part of the visible history.

Practical review guidance

Use the Git worksheet in `templates/GIT_LAB.md` inside a new disposable practice directory. Make three focused commits, implement a change on a branch, merge it and revert an intentionally bad commit. Before committing, inspect staged differences. Save the log, final rules, commands and an explanation of the staging area in `workspace/04_3_git_evidence.md`.

Run the supplied sandbox demonstration as a comparison, not as a substitute for your own exercise. Pass by identifying the exact content of each commit, explaining why the revert adds history and showing that your working tree is clean after completion.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 04.4: Review, documentation and responsible AI help

Core question 1

Question: What should a review criticize?

Answer and reasoning: The change, its behavior and evidence, not the author's character.

Core question 2

Question: Why is compiling AI-generated code insufficient validation?

Answer and reasoning: Compilation checks only some structural rules; logic, security, performance and requirements can still be wrong.

Core question 3

Question: What demonstrates understanding beyond copying a solution?

Answer and reasoning: Explaining the reasoning, testing edge cases and solving a related new task independently.

Application question 4

Question: What is the first requirement to clarify when reviewing the purchase candidate with balance=5 and cost=5?

Answer and reasoning: Whether exact affordability permits the purchase. Under the assigned rule it does, so a strict > comparison is incorrect at that boundary.

Application question 5

Question: Why is “the AI wrote it and all supplied tests pass” insufficient for this course's review check?

Answer and reasoning: You still need to understand the code, verify the intended contract, add relevant independent cases and explain assistance/provenance. Supplied tests cover only their stated cases.

Practical review guidance

Review a roughly ten-line change using `templates/CODE_REVIEW.md`. Address requirement, correctness, boundaries, side effects, clarity and tests. Produce a README for your simulator that another person can follow from a clean folder. Record assistance and provenance in the template.

Perform one small related change without copying the reference, predict its effects and update the tests. This is an evidence-based lab; there is no automatic function checker for it. Pass by explaining every changed line and by recording a clean-start result from another person or a clearly labeled self-review when independent review is not yet available.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Module 04: additional-exercise solutions

M04-E01

Exercise: Write a minimal double-damage report with one player and one event.

Worked answer or evidence standard: Include starting health 10, damage 3, expected 7, actual 4, exact command/source and the first divergent update.

M04-E02

Exercise: Create a correct function called twice by a buggy dispatcher.

Worked answer or evidence standard: The arithmetic remains correct but the event count violates one-command/one-update. Repair dispatch and add an integration test.

M04-E03

Exercise: Write a bounds test that fails an <=size mutant.

Worked answer or evidence standard: For size=4, starting position 3 and delta +1 must return 3, not accept 4.

M04-E04

Exercise: Write a progress test rejecting an always-no-op movement function.

Worked answer or evidence standard: An interior position 1 with delta +1 on size4 must become 2. A bounds-only property would miss this defect.

M04-E05

Exercise: Design a temporary-file test that never touches personal data.

Worked answer or evidence standard: Use a newly created temporary directory and cleanup scoped to that directory. Test valid and invalid fixtures there.

M04-E06

Exercise: Write one isolated test that does not rely on a prior test initializing state.

Worked answer or evidence standard: Construct its own starting state in the test or setup fixture, invoke one operation and assert its independent expected result.

M04-E07

Exercise: Stage one file version, edit it, and compare staged and unstaged differences.

Worked answer or evidence standard: The staged difference describes what will be committed; the unstaged difference describes subsequent edits not yet staged.

M04-E08

Exercise: Explain a revert using a four-commit history.

Worked answer or evidence standard: Baseline, feature, mistake and revert remain visible. The revert reverses the mistake without deleting its historical record.

M04-E09

Exercise: Review exact-affordability and zero-cost boundaries in a purchase change.

Worked answer or evidence standard: A cost equal to balance should reach zero; a zero cost should not change the balance. Invalid negative cost remains a separate rejection case.

M04-E10

Exercise: Write a provenance note for assisted code.

Worked answer or evidence standard: Name what assistance was used, which parts were accepted/changed, how they were verified and what the learner can explain independently. Do not disclose private material.

Module 04: review answers and project evaluation

M04-R01

Question: Why find the earliest state disagreement rather than only inspecting the final screen?

Answer: It narrows the defect to the first wrong operation and avoids trying to repair a later presentation symptom.

M04-R02

Question: What is an independent test oracle in this setting?

Answer: An expected answer derived from the specification, a hand calculation or a genuinely independent reference rather than duplicated implementation logic.

M04-R03

Question: What must a mutation experiment demonstrate?

Answer: The correct version passes, the intentional defect triggers a relevant failure, and the correct version is restored. State the mutation actually tested.

M04-R04

Question: Does git add always include edits made afterward?

Answer: No. Later edits need staging too if they are to enter the same next commit.

M04-R05

Question: What makes a README executable rather than merely descriptive?

Answer: It states prerequisites, working directory, exact commands, expected signals, controls, tests and known limitations that can be followed from a clean start.

Project acceptance guidance

The supplied defect is reproduced before its fix; a named test catches its reintroduction; the restored implementation passes the learner suite. Git history explains the changes. The README works from a clean extraction, and the learner can implement a related variation without copying.

Require the common rubric, essential failure checks and an explainable new variation. Independently observed results must be identified separately from self-reported results.

Module 05: A complete first game

Lesson 05.1: The update loop and command model

Core question 1

Question: Are a simulation tick and a rendered frame necessarily identical?

Answer and reasoning: No. Simulation and presentation can run at different rates.

Core question 2

Question: What should happen when render is called twice without an update?

Answer and reasoning: The simulation state should remain unchanged.

Core question 3

Question: Why is headless simulation useful?

Answer and reasoning: It enables automated tests, replays and server-style execution without a graphical interface.

Application question 4

Question: With an upper-left origin and y increasing downward, which coordinate change represents w/up?

Answer and reasoning: The delta is (0,-1): x stays the same and y decreases by one.

Application question 5

Question: Why should victory depend on the state rather than whether an E symbol remains visible after drawing?

Answer and reasoning: Rendering may hide E behind P or use another presentation. Game rules must use the actual goal and position, not infer state from one display representation.

Practical review guidance

Implement `normalise_command`, `step_position` and `render_grid` in `labs/student/lab_05_1.py`. The default board is 10x10; the helpers also accept a small square size for tests. Draw P at the player and E at the bottom-right exit unless the player occupies it. Unknown or out-of-bounds movement returns the unchanged valid position.

Connect these helpers in a terminal adapter in your workspace. Test one movement per input and repeated drawing with no updates. Compare a known 3x3 rendering, then run the actual 10x10 version. Pass by showing that the update can be called without input or printing and that the display is read-only.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 05.2: Collision rules and complete behavior

Core question 1

Question: What is missing from "move right increases x"?

Answer and reasoning: Boundary, obstacle, terminal-state and invalid-command policies, among other possible rules.

Core question 2

Question: Under the reject policy, what happens at x = 9 on a 0-9 grid?

Answer and reasoning: The right move is rejected and x stays 9.

Core question 3

Question: Why validate before applying a multi-part state change?

Answer and reasoning: It avoids leaving the system partially updated when one requirement fails.

Application question 4

Question: Why is changing x before checking whether the full candidate cell is blocked risky?

Answer and reasoning: A rejected action can leave a partial mutation. Validate the complete candidate and all affected fields before committing the transition.

Application question 5

Question: Under the capstone rule, what status results when the final energy unit is spent entering the goal?

Answer and reasoning: Won. Goal arrival has priority over energy exhaustion for that same accepted action; a dedicated test must preserve the stated policy.

Practical review guidance

Implement `step_world` in `labs/student/lab_05_2.py` with the fixed map, pure flat-state return and terminal win policy. Test all four edges, all five obstacles, a normal move, reaching the goal and commands after victory. Preserve the input dictionary and reject the complete move rather than changing one coordinate early.

Once the helper passes, implement the capstone energy/loss/restart requirements from `CAPSTONE_SPEC.md` in your own game. Keep separate evidence for the helper and the richer game. Pass by explaining collision rejection, terminal priority and a live restart that leaves no stale round state.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 05.3: Deterministic replay and simple measurement

Core question 1

Question: What information does a replay need besides button names?

Answer and reasoning: Initial state, order/tick placement, relevant configuration and controlled randomness/environment assumptions.

Core question 2

Question: Why exclude printing from a simulation microbenchmark?

Answer and reasoning: Otherwise the measurement may primarily reflect terminal I/O instead of simulation work.

Core question 3

Question: Does one matching replay prove determinism for all inputs?

Answer and reasoning: No. It is evidence for that case; broader tests and analysis are still required.

Application question 4

Question: Why can a replay with identical commands diverge after a map or rule change?

Answer and reasoning: The map and rules are part of the simulation conditions. A reproducible replay must identify them along with the initial state and command order.

Application question 5

Question: Why is a loop that reaches victory early and then performs thousands of no-op updates a misleading active-play benchmark?

Answer and reasoning: Most timed iterations exercise terminal-state rejection rather than normal movement and resource updates. The measured workload no longer represents the claimed operation.

Practical review guidance

Complete `labs/student/lab_05_3.py` after the Lesson 05.2 helper works. Replay an empty session, a twenty-command session and the known route to the goal. Compare both state fields and canonical digests across repeated runs. Preserve exact commands and rule assumptions.

Run `tools/benchmark_game.py` as a reference experiment, then adapt the same method to your own implementation without comparing unlike workloads. Keep the raw JSON report and state which operations are included. Pass by reproducing a session and explaining why the microbenchmark is not a whole-game FPS measurement.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Lesson 05.4: Finishing, usability and the first portfolio artifact

Core question 1

Question: What belongs in a clean-start README?

Answer and reasoning: Prerequisites, exact run/test commands, controls, expected behavior and known limitations.

Core question 2

Question: Why should color not be the only important cue?

Answer and reasoning: Some players cannot reliably distinguish it; a second channel such as text or shape preserves the information.

Core question 3

Question: What evidence should accompany a claimed improvement?

Answer and reasoning: A reproducible comparison with the same workload and clearly stated measurement conditions.

Application question 4

Question: The supplied reference game passes its tests. Does copying it unchanged satisfy the student Gate A implementation requirement?

Answer and reasoning: No. The learner must implement and explain their own artifact or make substantial independently understood changes, write learner-owned tests and complete the review evidence.

Application question 5

Question: A tester needs verbal instructions that are absent from the README. What should the release review record?

Answer and reasoning: Record the missing instruction as a reproducibility/usability defect, improve the documentation or interface, and repeat the clean-start check instead of pretending the original instructions were sufficient.

Practical review guidance

Complete `CAPSTONE_SPEC.md` and the five-module project requirements. Build your own game under `capstone/student/` or an explicitly documented workspace. Replace the intentionally failing starter test with at least twelve independent tests and add a meaningful design change rather than merely submitting the reference.

Ask a tester to follow your README without coaching, record three observations and repair at least one issue. Submit the release evidence and demonstrate a live rule variation. The supplied game checker tests the reference, not your project; use your own test command and independent review for Gate A.

Reviewer check: ask for the changed-case challenge without copying the walkthrough. Inspect the saved prediction and the actual implementation or worksheet. A reference-mode test log is not evidence that the student files passed.

Module 05: additional-exercise solutions

M05-E01

Exercise: Write the four coordinate deltas for the declared screen-grid convention.

Worked answer or evidence standard: w=(0,-1), a=(-1,0), s=(0,1), d=(1,0).

M05-E02

Exercise: Show two render calls with no intervening update and compare state.

Worked answer or evidence standard: Both renderings agree for the same state, and all state fields are unchanged. A display operation cannot consume energy or turns.

M05-E03

Exercise: Construct a blocked movement at each edge.

Worked answer or evidence standard: Use x=0/a, x=9/d, y=0/w and y=9/s with valid open cells; assert complete state equality before and after.

M05-E04

Exercise: Construct a wall collision that would expose partial energy deduction.

Worked answer or evidence standard: From (1,0), s attempts obstacle (1,1). Position, energy, turns and status must all remain unchanged.

M05-E05

Exercise: Predict the known win route: nine d then nine s.

Worked answer or evidence standard: End at (9,9), energy7, turns18, statuswon. The top and right edges avoid all fixed obstacles.

M05-E06

Exercise: Predict seven waits followed by the win route.

Worked answer or evidence standard: There are 25 accepted actions; final energy0, turns25 and statuswon because goal arrival wins the tie.

M05-E07

Exercise: Explain how a successful load affects recording in the reference interface.

Worked answer or evidence standard: It begins a new replay segment with the loaded state as initial state and clears the old segment commands, preventing mismatched initial conditions.

M05-E08

Exercise: Compare canonical state serialization with a screenshot.

Worked answer or evidence standard: Canonical serialization directly represents specified state fields in a defined order/encoding; a screenshot can omit or obscure state. Neither alone proves transition correctness.

M05-E09

Exercise: Design a benchmark avoiding terminal-state no-op dominance.

Worked answer or evidence standard: Use a documented repeatable cycle containing active moves and restarts, verify final states, run repeated timed trials and exclude unrelated display/file work if claiming a headless measure.

M05-E10

Exercise: Propose a meaningful personal extension and its regression risks.

Worked answer or evidence standard: Examples: configurable controls or a new deterministic map. State changed rules, new tests, compatibility implications and unchanged invariants; avoid expanding scope before required behavior works.

Module 05: review answers and project evaluation

M05-R01

Question: Which fields remain unchanged on a rejected wall movement in the capstone?

Answer: Position, energy, turn count and status all remain unchanged. A diagnostic message may be different, but it is not persisted simulation state.

M05-R02

Question: How is the win-versus-exhaustion tie resolved?

Answer: Entering the goal wins, even when energy becomes zero on that same accepted action.

M05-R03

Question: What belongs in a reproducible replay besides commands?

Answer: The initial state, supported replay-format version, rule/map identifier and relevant environment/implementation assumptions.

M05-R04

Question: Why is the supplied benchmark not a frames-per-second result?

Answer: It times a specified headless Python loop including update overhead, not the complete input/simulation/render/presentation workload of a real-time game.

M05-R05

Question: What distinguishes a finished student artifact from a supplied demonstration?

Answer: Independent implementation and understanding, learner-authored tests, documented changes, replay and review evidence, clean-start usability, and a live variation the learner can defend.

Project acceptance guidance

All essential Gate A rules pass the student-owned tests and manual review. The tester can start, play, encounter rejection, win or lose, restart, save/load, export/replay, quit and rerun tests. A copied reference plus its test log does not pass.

Require the common rubric, essential failure checks and an explainable new variation. Independently observed results must be identified separately from self-reported results.